

HDOC Day-39 — Program 1

Name- Aaryan Pratap Singh

Sap Id - 590032842

Variadic Function for Weighted Average

Algorithm

1. Start.
2. Define a variadic function `weightedAverage(int count, ...)`.
3. Initialize `weightedSum = 0` and `totalWeight = 0`.
4. Start processing variable arguments using `va_start`.
5. Repeat `count` times:
 - Read a `value` as `double`.
 - Read its corresponding `weight` as `double`.
 - If the weight is negative, display an error message, terminate argument processing, and return an error value.
 - Add `value * weight` to `weightedSum`.
 - Add `weight` to `totalWeight`.
6. End variable argument processing using `va_end`.
7. If `totalWeight` is zero, display an appropriate error message and return an error value.
8. Calculate `weightedSum / totalWeight`.
9. Return the weighted average.
10. In `main()`, call the function with the required value-weight pairs.
11. Display the weighted average.
12. Stop.

Test Case Table

Test Case	Input	Expected Output
1	<code>weightedAverage(3, 80.0,0.3, 75.0,0.2, 90.0,0.5)</code>	84.00
2	<code>weightedAverage(2, 50.0,0.5, 100.0,0.5)</code>	75.00
3	<code>weightedAverage(2, 50.0,-0.5, 80.0,1.5)</code>	Error: Negative
4	<code>weightedAverage(2, 50.0,0.0, 80.0,0.0)</code>	Error: Total weight is zero
5	<code>weightedAverage(1, 90.0,1.0)</code>	90.00

C Program

```
#include <stdio.h>

#include <stdarg.h>

double weightedAverage(int count, ...)
{
    va_list args;

    double value, weight;

    double weightedSum = 0.0;

    double totalWeight = 0.0;

    int i;

    if (count <= 0)
    {
        printf("Error: Count must be positive.\n");

        return -1.0;
    }

    va_start(args, count);

    for (i = 0; i < count; i++)
    {
        value = va_arg(args, double);

        weight = va_arg(args, double);

        if (weight < 0)
        {
            printf("Error: Weights cannot be negative.\n");

            va_end(args);

            return -1.0;
        }
    }
```

```

    weightedSum += value * weight;

    totalWeight += weight;
}

va_end(args);

if (totalWeight == 0.0)
{
    printf("Error: Total weight cannot be zero.\n");

    return -1.0;
}

return weightedSum / totalWeight;
}

int main()
{
    double result;

    result = weightedAverage(3,
                            80.0, 0.3,
                            75.0, 0.2,
                            90.0, 0.5);

    if (result >= 0.0)
    {
        printf("Weighted Average = %.2f\n", result);
    }

    return 0;
}

```

EXECUTION

```
[aaryanrajput@aaryans-macbook ~ % nano day39p1.c  
[aaryanrajput@aaryans-macbook ~ % clang day39p1.c -o day39p1  
[aaryanrajput@aaryans-macbook ~ % ./day39p1  
Weighted Average = 84.00  
aaryanrajput@aaryans-macbook ~ % █
```

HDOC Day-39 — Program 2

Command-Line Arguments — Integer Statistics

This program accepts any number of arguments such as:

It ignores invalid arguments like `abc`.

Algorithm

1. Start.
2. Obtain command-line arguments using `argc` and `argv`.
3. For every argument from `argv[1]` onwards:
 - Check whether it represents a valid integer.
 - If valid, convert it into an integer and store it.
 - Otherwise, display `Invalid argument ignored`.
4. If no valid integers are available, display an error and terminate.
5. Sort the valid integers in ascending order.
6. Display the sorted integers.
7. The first sorted element is the minimum.
8. The last sorted element is the maximum.
9. Calculate the sum and divide it by the number of valid integers to obtain the average.
10. Find the median:
 - If the number of integers is odd, select the middle element.
 - If even, find the average of the two middle elements.
11. Starting from the second-last element, search backwards for the first value different from the maximum. This is the second-largest distinct value.
12. If no such value exists, display an appropriate message.
13. Stop.

Test Case Table

Test t	Command-Line Arguments	Expected Result
1	12 5 abc 8 12 20 3	Ignore abc, sorted: 3 5 8 12 12 20
2	1 2 3 4 5	Min=1, Max=5, Avg=3, Median=3, Second largest=4
3	10 20 30 40	Median=25
4	abc xyz	No valid integers
5	5 5 5	No second-largest distinct value
6	-10 5 -2 20	Sorted: -10 -2 5 20
7	10 abc 20 xyz 30	Ignore abc and xyz

C Program

```
#include <stdio.h>

#include <stdlib.h>

#include <ctype.h>

#include <limits.h>

#include <errno.h>

/* Check whether a string represents a valid int */

int isValidInteger(char str[])
{
    char *end;

    long value;

    if (str[0] == '\0')
        return 0;

    errno = 0;

    value = strtol(str, &end, 10);

    if (*end != '\0')
        return 0;
```

```

    if (errno == ERANGE || value < INT_MIN || value > INT_MAX)

        return 0;

    return 1;
}

/* Sort integers in ascending order */

void sortArray(int a[], int n)
{
    int i, j, temp;

    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (a[j] > a[j + 1])
            {
                temp = a[j];

                a[j] = a[j + 1];

                a[j + 1] = temp;
            }
        }
    }
}

/* Calculate average */

double findAverage(int a[], int n)
{
    long long sum = 0;

    int i;

```

```

    for (i = 0; i < n; i++)

        sum += a[i];

    return (double)sum / n;
}

/* Calculate median of a sorted array */

double findMedian(int a[], int n)

{
    if (n % 2 != 0)

        return a[n / 2];

    return ((double)a[n / 2 - 1] + a[n / 2]) / 2.0;
}

/* Find second-largest distinct value */

int findSecondLargest(int a[], int n, int *found)

{
    int i;

    int largest = a[n - 1];

    for (i = n - 2; i >= 0; i--)

    {
        if (a[i] != largest)

        {
            *found = 1;

            return a[i];
        }
    }

    *found = 0;

    return 0;
}

```

```

}

int main(int argc, char *argv[])
{
    int *numbers;

    int count = 0;

    int i;

    int found;

    int secondLargest;

    double average, median;

    if (argc == 1)
    {
        printf("Error: No command-line arguments provided.\n");

        return 1;
    }

    numbers = malloc((argc - 1) * sizeof(int));

    if (numbers == NULL)
    {
        printf("Error: Memory allocation failed.\n");

        return 1;
    }

    /* Validate command-line arguments */

    for (i = 1; i < argc; i++)
    {
        if (isValidInteger(argv[i]))
        {
            numbers[count] = (int)strtol(argv[i], NULL, 10);

```

```

        count++;

    }

    else

    {

        printf("Invalid argument ignored: %s\n", argv[i]);

    }

}

if (count == 0)

{

    printf("Error: No valid integers provided.\n");

    free(numbers);

    return 1;

}

/* Sort valid integers */

sortArray(numbers, count);

printf("Valid integers in ascending order: ");

for (i = 0; i < count; i++)

{

    printf("%d", numbers[i]);

    if (i < count - 1)

        printf(" ");

}

printf("\n");

/* Minimum and maximum */

printf("Minimum: %d\n", numbers[0]);

printf("Maximum: %d\n", numbers[count - 1]);

```

```

/* Average */

average = findAverage(numbers, count);

printf("Average: %.2f\n", average);

/* Median */

median = findMedian(numbers, count);

printf("Median: %.2f\n", median);

/* Second-largest distinct value */

secondLargest = findSecondLargest(numbers, count, &found);

if (found)

    printf("Second-largest distinct value: %d\n",

        secondLargest);

else

    printf("Second-largest distinct value does not exist.\n");

free(numbers);

return 0;

}

```

EXECUTION

```

[aaryanrajput@aaryans-macbook ~ % ./day39p2.c 12 5 abc 8 12 20 3
Invalid argument ignored: abc
Valid integers in ascending order: 3 5 8 12 12 20
Minimum: 3
Maximum: 20
Average: 10.00
Median: 10.00
Second-largest distinct value: 12
aaryanrajput@aaryans-macbook ~ %

```